

DL LAB 1 – Fully Connected Networks on Fashion-MNIST

Thanks to:

Po-Kai Yang, Martina Montenegro, Juha Carlon,
Ehsan Moravveji, Bart Vanrumste



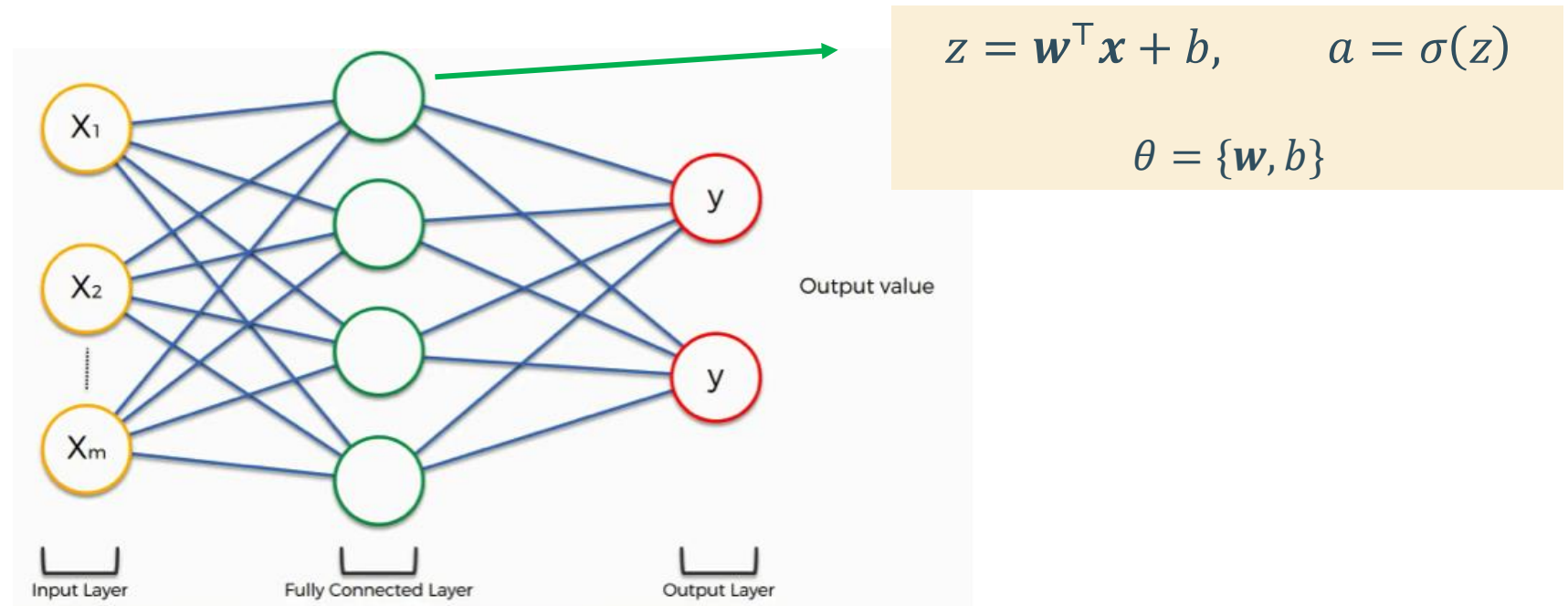
Agenda

- Brief Theory Recap
- Dataset
- Setup VSC for coding
- Code and questions: *LAB1_DL_2627_ASSIGNMENT.PDF* (solutions will be uploaded on TOLEDO)
- GPU use introduction

LAB 1 - Theory Recap

Feed-Forward Neural Network (FFNN)

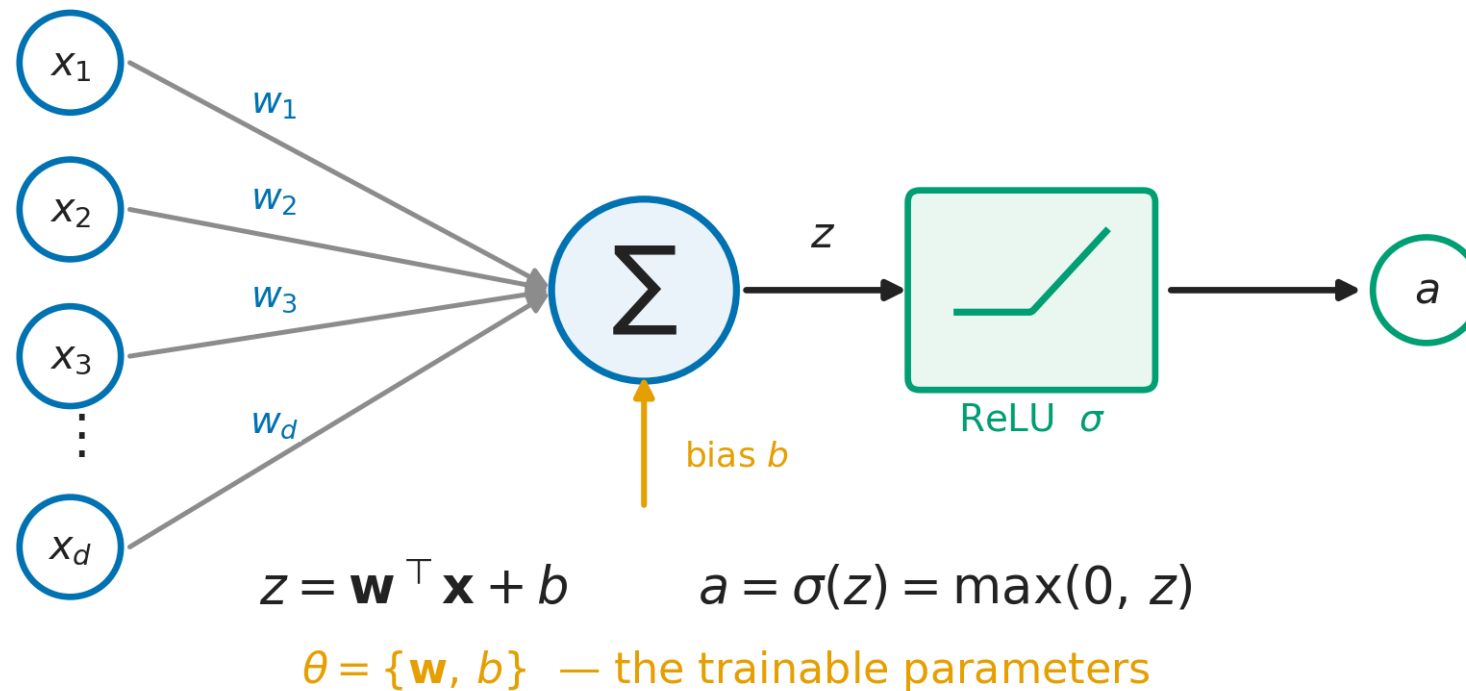
Also called Multi-Layer Perceptron (MLP) or Fully-Connected Network (FCN). It maps an input vector $\mathbf{x}$ to an output $\mathbf{y}$, by passing it through a sequence of layers.



* MLP has no spatial awareness

One neuron

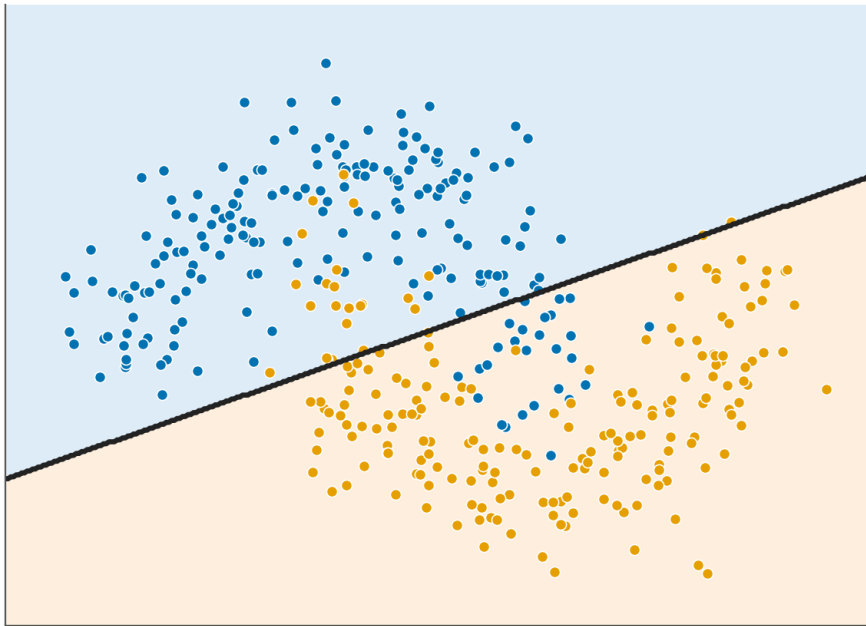
One neuron: weighted sum, bias, non-linearity



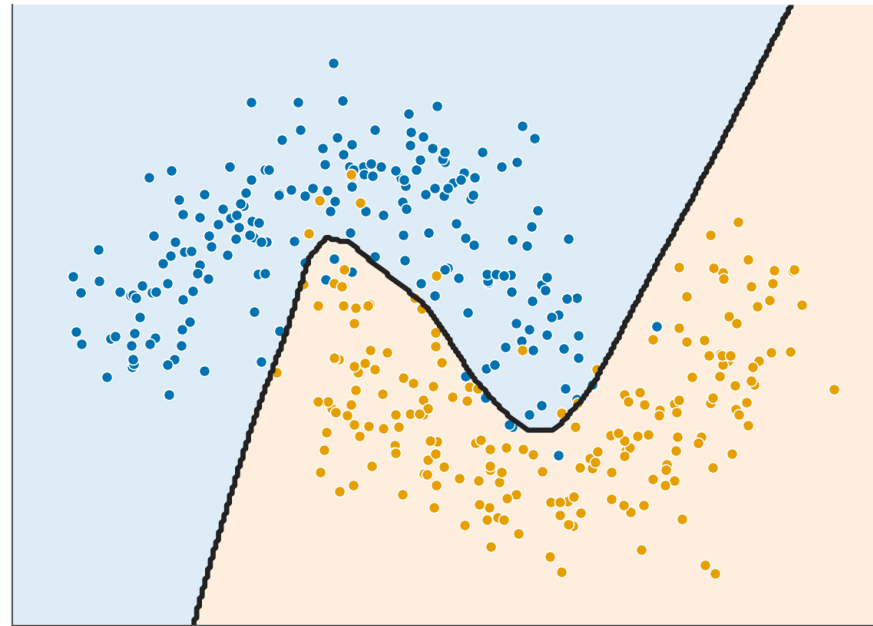
Why we add non-linearity

Without a non-linearity the boundary can only be a straight line

**Linear model
(no hidden layer)
accuracy 87%**



**MLP with ReLU
(2 hidden layers)
accuracy 96%**

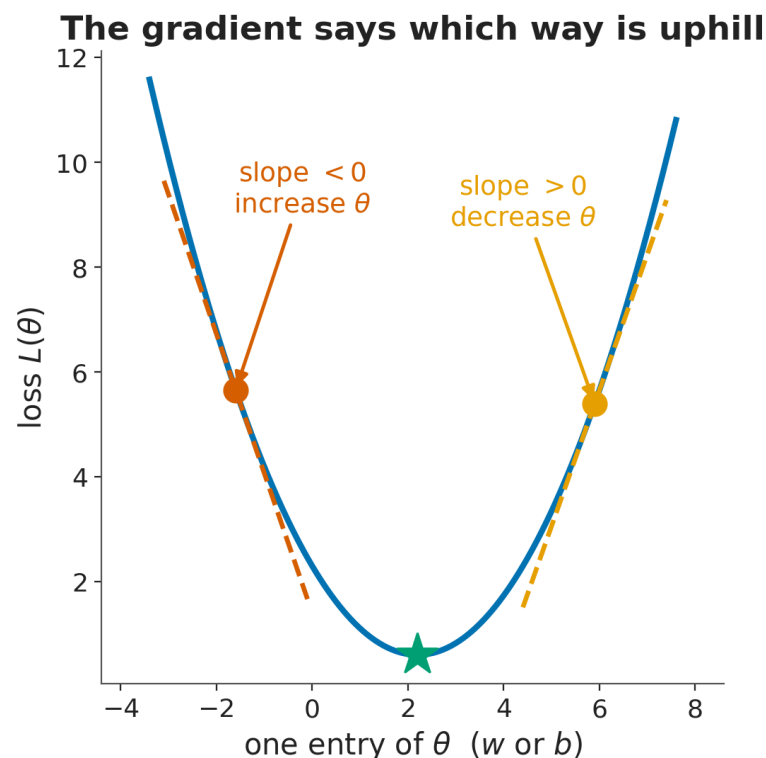
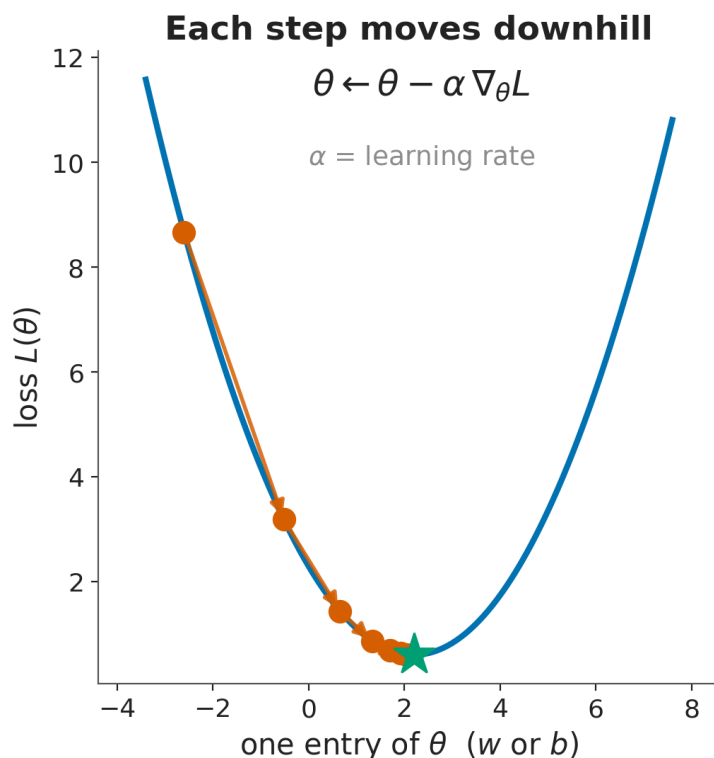


Gradient Descent

- **Training** means adjusting θ to minimise a **loss function** $L(\theta)$ that quantifies how wrong the predictions are.

$$\nabla_{\theta} L = \partial L / \partial \theta$$

Gradient descent



* Sign explained:

If the slope is negative, moving to the right reduces the loss, so the update moves right.

If the slope is positive, moving to the left reduces the loss, so the update moves left.

In both cases, the update points towards the minimum.

$\theta = \{w, b\}$ — every trainable number: the weights w and the biases b .
The 1-D axis is one of them. The same update is applied to all of them.

Learning Rate

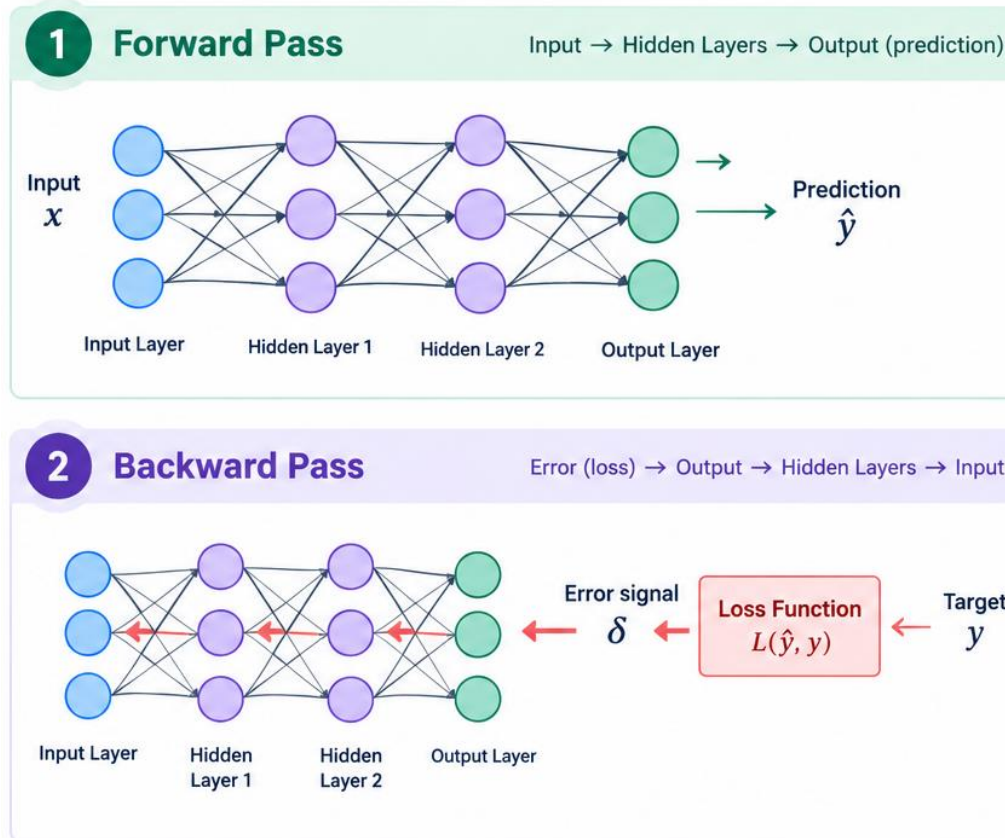
The learning rate controls the size of the update in basic gradient descent.

α	Typical behaviour
far too small	loss decreases, but so slowly that 20 epochs are not enough
good	fast, smooth decrease, settling into a low plateau
slightly too large	loss decreases but the curve is visibly jagged
far too large	loss oscillates, plateaus at a high value, or becomes NaN

There is no universal best value: it depends on the optimizer (Adam tolerates a much narrower range around 10^{-3} ; plain SGD often wants 10^{-1}), the input scale, and the batch size.

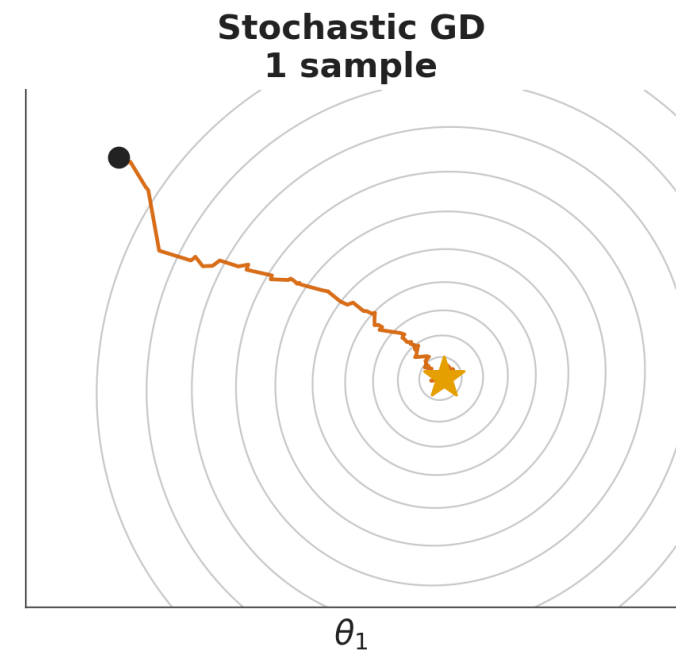
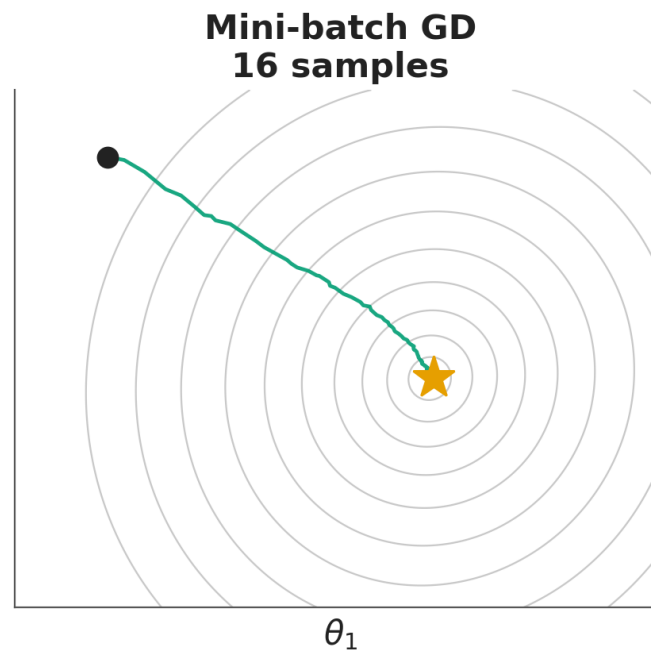
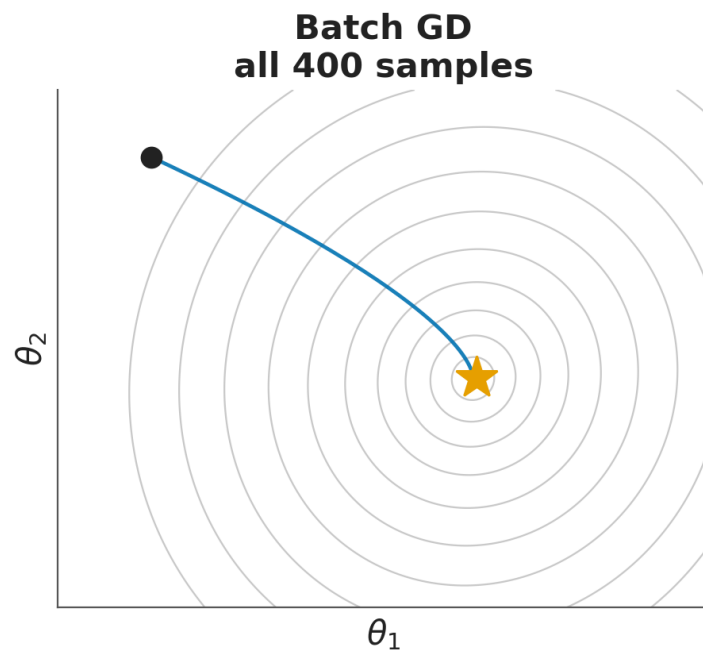
Backpropagation

Backpropagation is the algorithm that computes this gradient efficiently



Batch, stochastic and mini-batch Gradient Descent

Method	Data per update	Gradient estimate	Practical use
Batch GD	the entire training set (N samples)	exact, no noise	fine for small data; far too slow at $N = 50,000$
Stochastic GD	one sample	very noisy, very cheap	rarely used alone today
Mini-batch GD	a group of B samples	noisy but stable enough	the standard in deep learning



Example.... Batch Size

The number of weight updates per epoch is

$$\text{updates per epoch} = \left\lceil \frac{N}{B} \right\rceil$$

In this lab $N = 50,000$ training images, so `batch_size=64` gives $\lceil 50000/64 \rceil = 782$ updates per epoch.

* **Terminology trap:** PyTorch's `torch.optim.SGD` is named after *stochastic* gradient descent, but because you always feed it mini-batches from a `DataLoader`, what you are actually running is **mini-batch** gradient descent.

```
OPTIMIZER = "SGD" //          BATCH_SIZE = 64  # step 12 – true SGD would be BATCH_SIZE = ....

if name == "SGD": return torch.optim.SGD(model.parameters(), lr=lr)

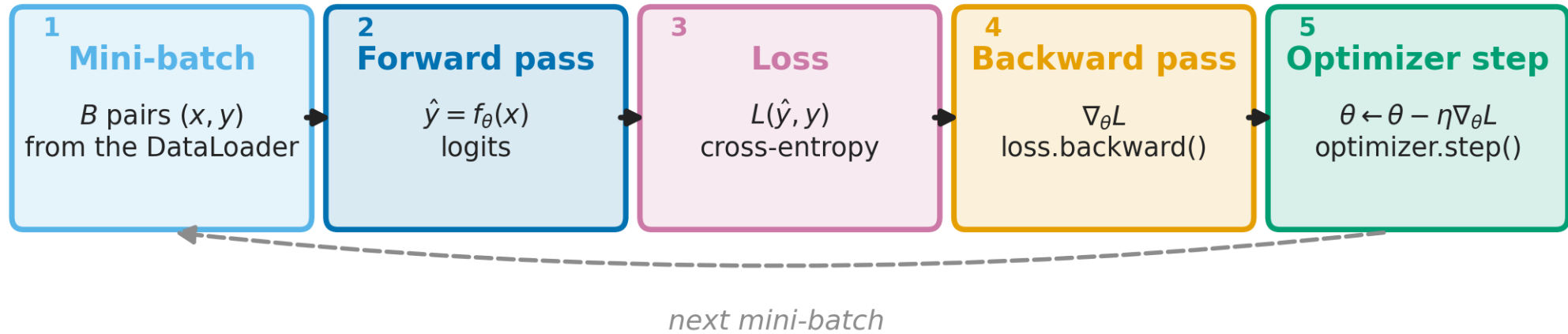
train_loader = DataLoader(train_ds, batch_size=batch_size, shuffle=True)

for features, labels in loader:
    features, labels = features.to(device), labels.to(device)
    outputs = model(features)
    loss = loss_fn(outputs, labels)
```

shuffle = True: the batch composition is re-randomised every epoch.

One weight update

`optimizer.zero_grad()` clears the gradients before each new batch



repeat for every mini-batch → one epoch → repeat for many epochs

This is repeated for the x number of updates to complete one epoch.

- **Logits:** raw scores. Example: for a classification problem with 3 classes the output can be = $[2.5, 0.3, -1.2] \rightarrow 3$ logits

Example...

for x, y in train_loader:

1. Mini-batch: clear old gradients
`optimizer.zero_grad()`

2. Forward pass
`logits = model(x)`

3. Calculate the loss
`loss = criterion(logits, y)`

4. Backward pass: calculate gradients
`loss.backward()`

5. Optimizer step: update parameters
`optimizer.step()`

PyTorch code

`optimizer.zero_grad()`

`logits = model(x)`

`loss = criterion(logits, y)`

`loss.backward()`

`optimizer.step()`

What happens

Clears previous gradients.

Calculates the model outputs.

Measures prediction error.

Computes gradients.

Updates model parameters.

Output of the model

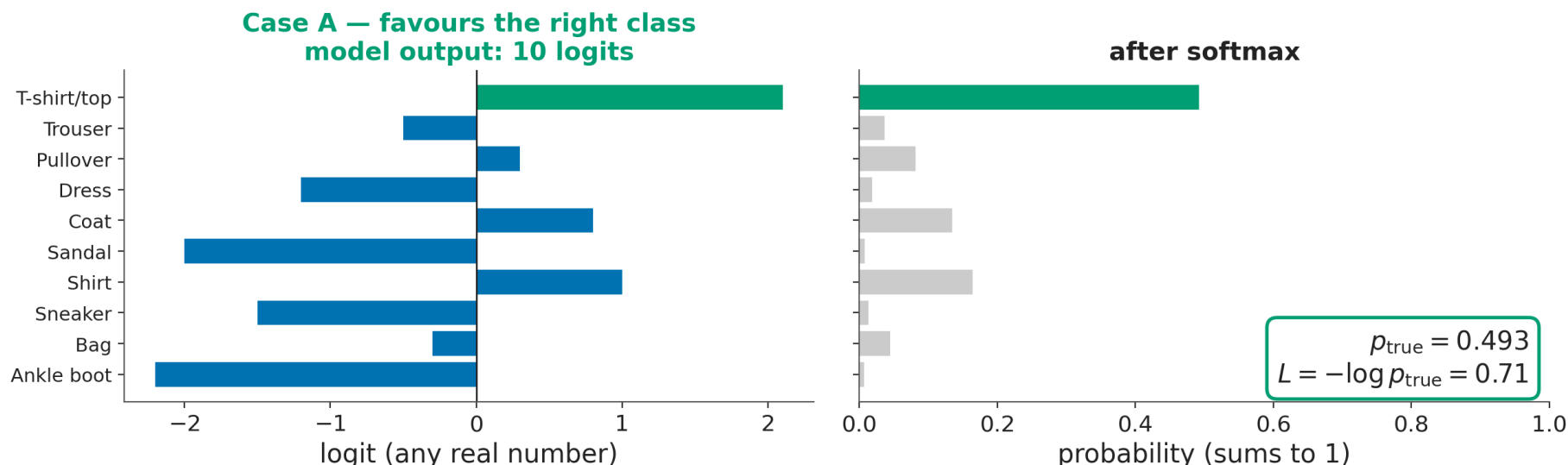
- The final Linear layer produces **10 numbers per image**, one score per class. These scores are called **logits**

Class:	T-shirt	Trouser	Pullover	Dress	Coat	Sandal	Shirt	Sneaker	Bag	Boot
Logit:	2.1	-0.5	0.3	-1.2	0.8	-2.0	1.0	-1.5	-0.3	-2.2

Suppose the true class is T-shirt and the T-shirt logit is 2.1.

- nn.CrossEntropyLoss**: apply Softmax (turns logits into probabilities) $p_j = \frac{e^{z_j}}{\sum_{k=1}^{10} e^{z_k}}$ and negative log-likelihood (read off the probability assigned to the *true* class and take its negative logarithm) $L = -\log p_{y_{\text{true}}}$

True class = T-shirt/top. CrossEntropyLoss applies softmax internally — so the model outputs logits and has no Softmax layer.



Optimizers

- **SGD:** applies the raw gradient, scaled by the learning rate

$$\theta_{t+1} = \theta_t - \alpha g_t, \quad g_t = \nabla_{\theta} L_b \text{ (mini-batch estimate of } \nabla_{\theta} L \text{)}$$

- **SGD + momentum:** accumulates a velocity vector (Consistent gradient directions reinforce each other and build speed, while oscillating components cancel out.

$$v_{t+1} = \beta v_t + (1 - \beta) g_t, \quad \theta_{t+1} = \theta_t - \alpha v_{t+1}$$

- **RMSprop:** parameters with consistently large gradients get small steps and vice versa, which largely removes the sensitivity to the overall scale of the gradients.

$$s_{t+1} = \beta s_t + (1 - \beta) g_t^2, \quad \theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{s_{t+1}} + \epsilon} g_t$$

- **Adam:** combines both ideas: momentum on the gradient (first moment m_t) and RMSprop's per-parameter scaling (second moment v_t), plus a bias correction that compensates for both running averages starting at zero

Batch Normalization – optimization aid

- **BatchNorm** normalises each feature using statistics computed over the current mini-batch, then lets the network learn its own scale and shift:

$$\bar{z} = \frac{z - \mu_{\text{batch}}}{\sqrt{\sigma_{\text{batch}}^2 + \epsilon}}, \quad \tilde{z} = \gamma \bar{z} + \beta$$

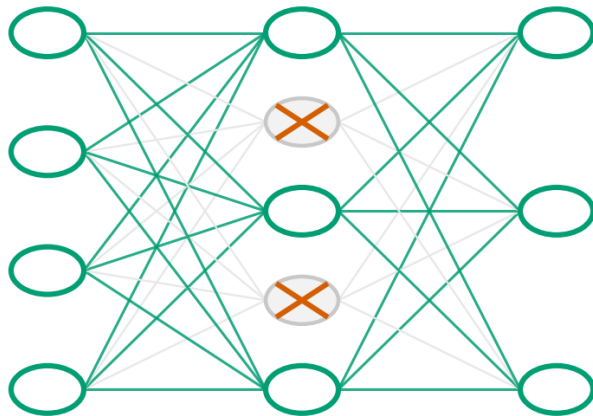
Symbol	Role
$\mu_{\text{batch}}, \sigma_{\text{batch}}^2$	mean and variance over the current mini-batch, per feature
γ, β	learnable scale and shift, per feature
ϵ	small constant for numerical stability

* Note that standardising the inputs does the same thing for the *first* layer only; BatchNorm maintains that property at every hidden layer throughout training.

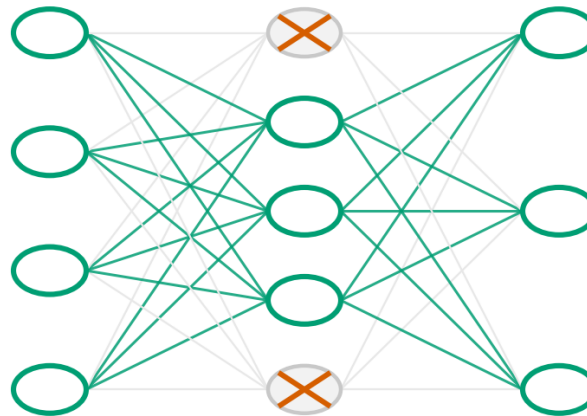
Dropout - regularization

Dropout: neurons cannot rely on any specific partner being present

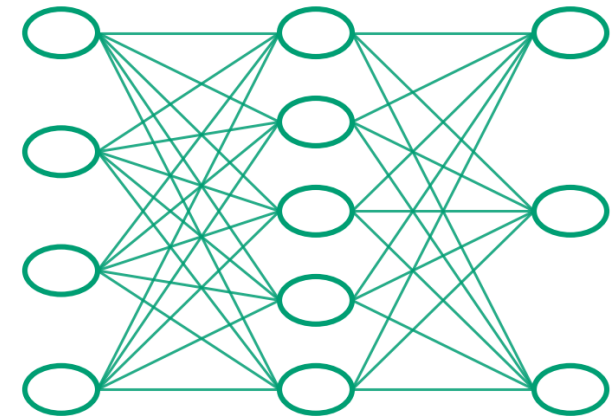
Training step 1
(two of five units dropped)



Training step 2
(a different two)



Evaluation — `model.eval()`
(full network, dropout off)



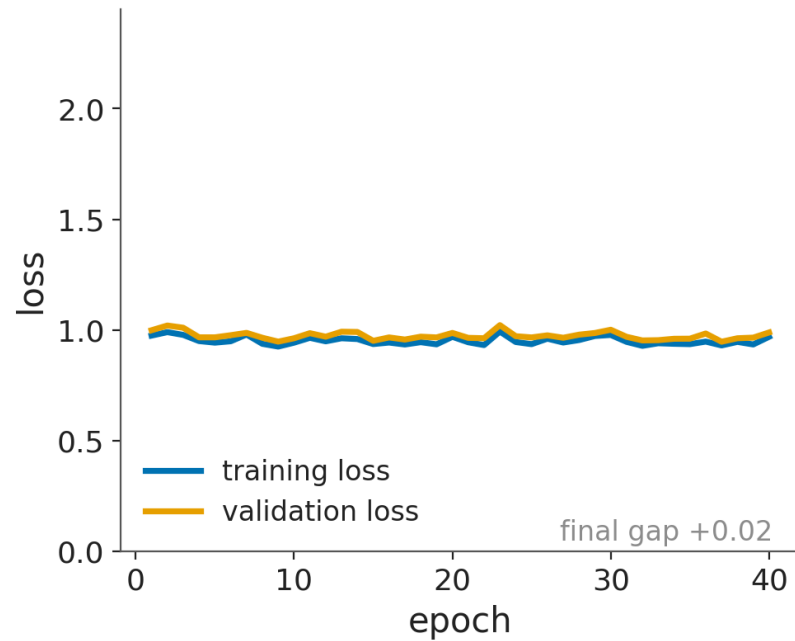
Training: each activation is zeroed with probability p , survivors scaled by $1/(1 - p)$ so the expected output is unchanged. That is why evaluation simply switches dropout off — no rescaling needed. Expect training loss to RISE and validation loss to FALL.

The standard block is Linear → BatchNorm → ReLU → Dropout

Train and Validation Loss

Underfitting

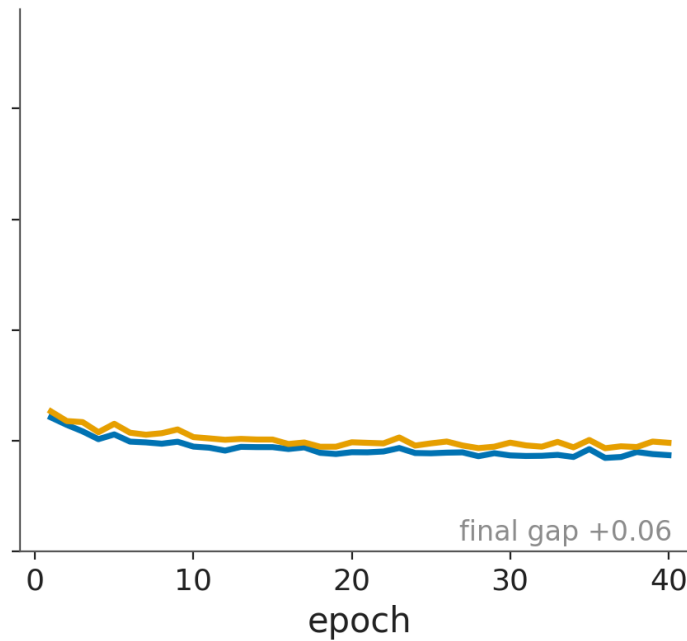
too little capacity



both losses stay high
→ more capacity, train longer

Good fit

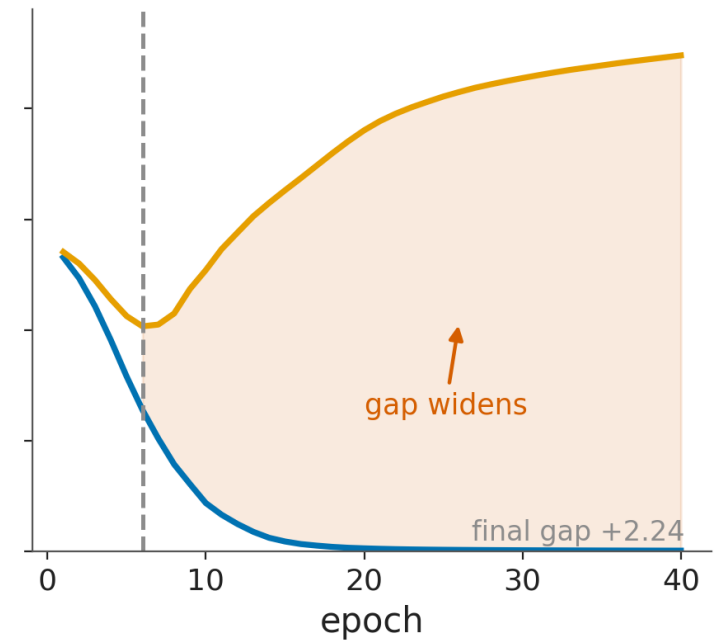
enough capacity, enough data



both low, small gap
→ this is what you want

Overfitting

lots of capacity, little data



train ↓ while val ↑
→ regularise or stop earlier

Early Stopping

- Instead of training for a fixed number of epochs, it automatically stops training when the validation metric no longer improves. This helps prevent overfitting and saves computational resources.

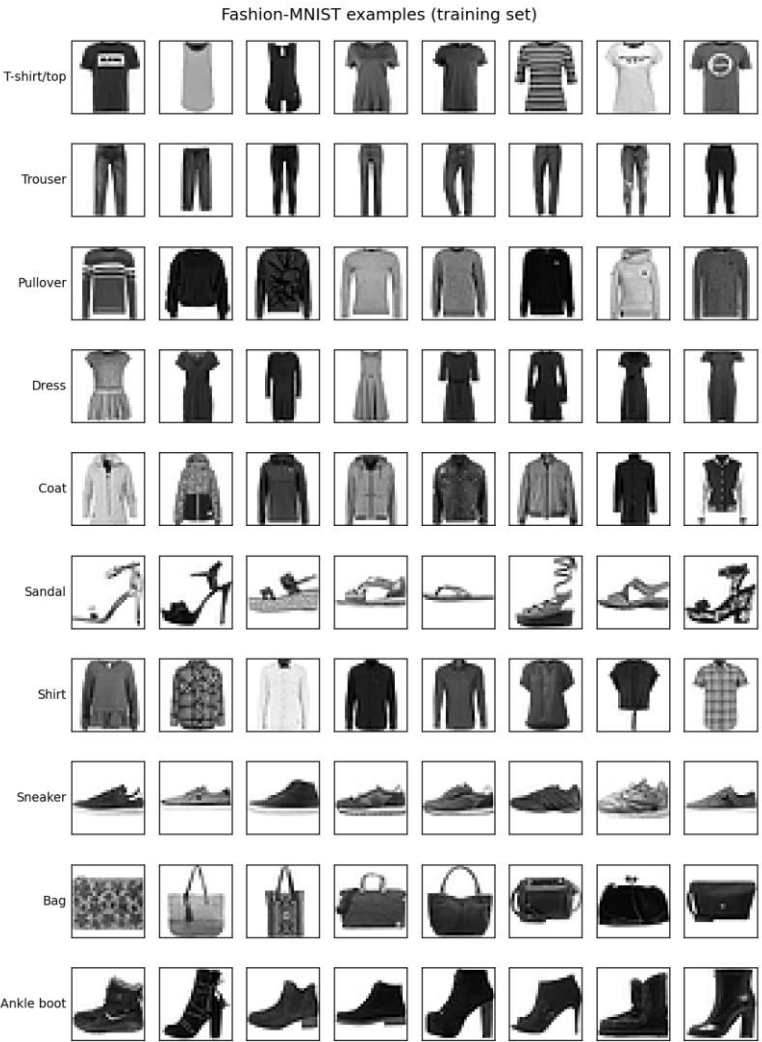


`restore_best=True` reloads the best epoch's weights before saving and testing.

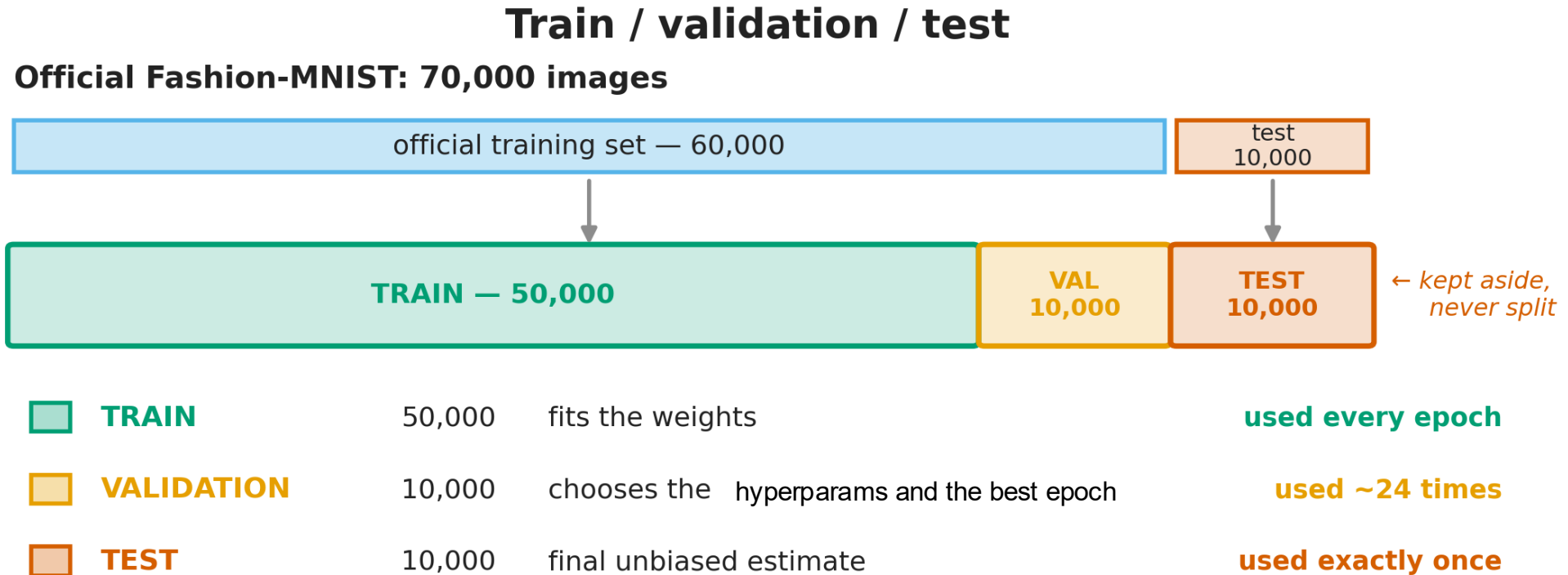
Dataset

Fashion-MNIST

Property	Value
Images	70,000 grayscale, 28×28 pixels
Official split	60,000 train / 10,000 test
Pixel values	integers 0–255 (0 = white background, 255 = fully dark ink)
Features per image after flattening	28×28 = 784
Classes	10, perfectly balanced (6,000 train images each) 0 = T-shirt/top 3 = Dress 6 = Shirt 9 = Ankle boot 1 = Trouser 4 = Coat 7 = Sneaker 2 = Pullover 5 = Sandal 8 = Bag



Dataset split



Every choice made by looking at a score biases that score.
The test set stays sealed until step 16 so the final number is honest.

Standardization

- Fit the mean and std on the training set only
- Neural networks train more reliably when input features are on comparable scales. **Z-score standardisation** rescales each feature to mean 0, standard deviation 1

$$x' = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}}$$

VSC OnDemand

FOR COPYING THE COMMANDS....

- Download from TOLEDO: **LAB1_DL_2627_ASSIGNMENT.PDF**
- In the file there are the command lines to copy and paste in the terminal, needed to access the dataset and prepare the environment to run today lab code

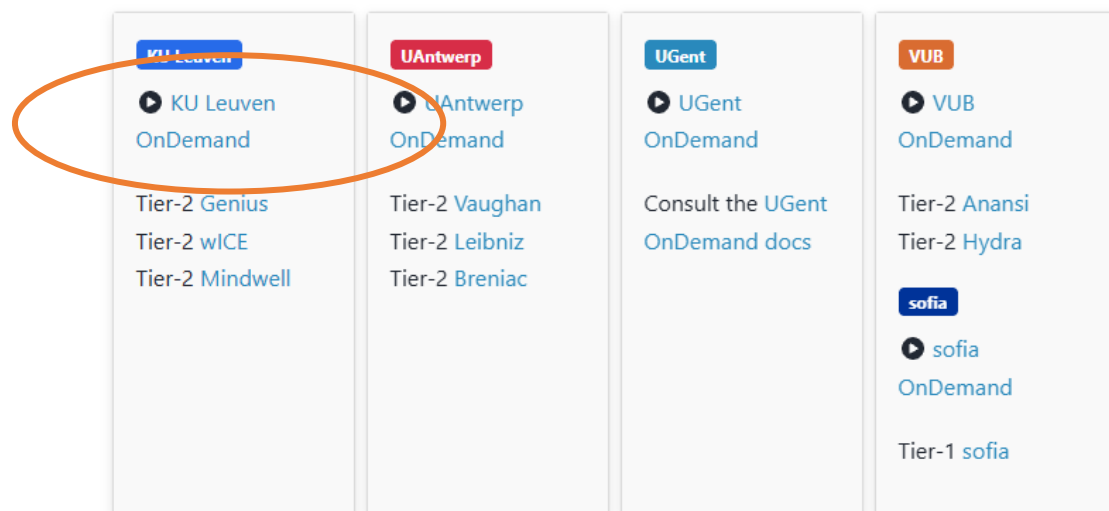
0. Access

- Open OnDemand web portal provides a user interface to HPC clusters from within a web browser.

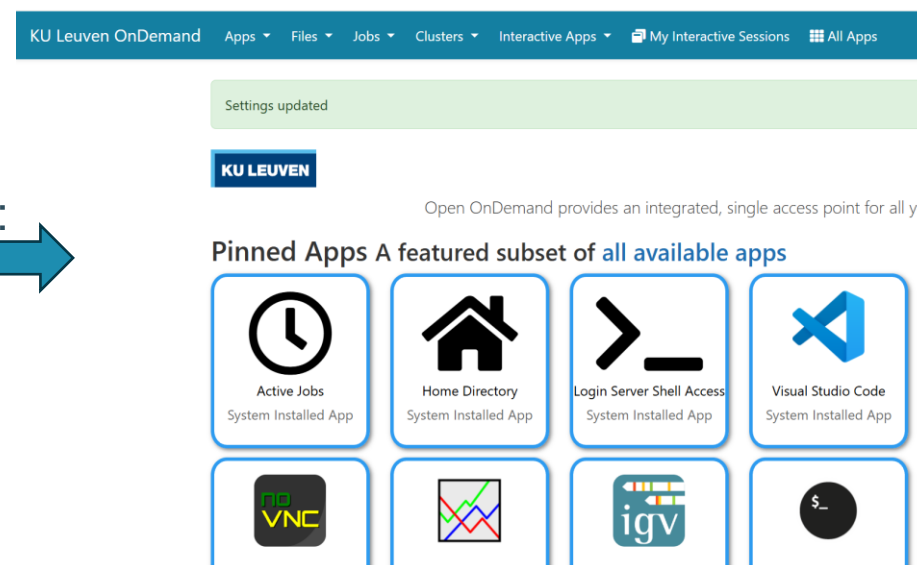
<https://docs.vscentrum.be/compute/portal/index.html>

VSC OnDemand Portals

VSC clusters that support an Open OnDemand web portal:



After log in:

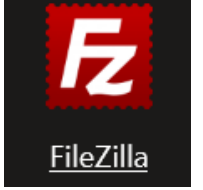


- <https://ondemand.hpc.kuleuven.be/pun/sys/dashboard> (in case the other link fails)

DATA UPLOAD

(for your own project)

1. FileZilla



- Data transfer program needed : FileZilla: [Data transfer with FileZilla — VSC documentation](#)

Note: less than 1.5GB can usually be uploaded more easily as shown later

2. OnDemand: you can upload smaller files

<https://auth.vscentrum.be/auth/login?mfa=true>

The screenshot displays the KU Leuven OnDemand web interface. At the top, a dark blue navigation bar contains the text 'KU Leuven OnDemand' followed by several menu items: 'Apps', 'Files', 'Jobs', 'Clusters', 'Interactive Apps', and 'My Interactive Sessions'. The 'Files' menu is currently open, showing a dropdown with 'Home Directory' and 'Data Directory /data/leuven/378/vsc37872'. Below this, a light green bar represents the file system view, with a 'Data Directory' label. At the bottom of the interface, a horizontal toolbar contains several action buttons: 'Open in Terminal', 'Refresh', 'New File', 'New Directory', 'Upload', 'Download', 'Globus', 'Copy/Move', and 'Delete'. A blue arrow points from the bottom right towards the 'Upload' button, highlighting it.

Files -> Data Directory

bs ▾ Clusters ▾ Interactive Apps ▾ My Interactive Sessions ? Help ▾ Logged in as vsc37872 [Log Out](#)

[_Open in Terminal](#) [Refresh](#) [+ New File](#) [New Directory](#) [Upload](#) [Download](#) [Open in Globus](#) [Copy/Move](#) [Delete](#)

[↑](#) / data / leuven / 378 / vsc37872 / [Change directory](#) [Copy path](#)

☐ Show Owner/Mode ☐ Show Dotfiles Filter:

Showing 13 of 16 rows - 0 rows selected

Create new directory: **dl-lab1**

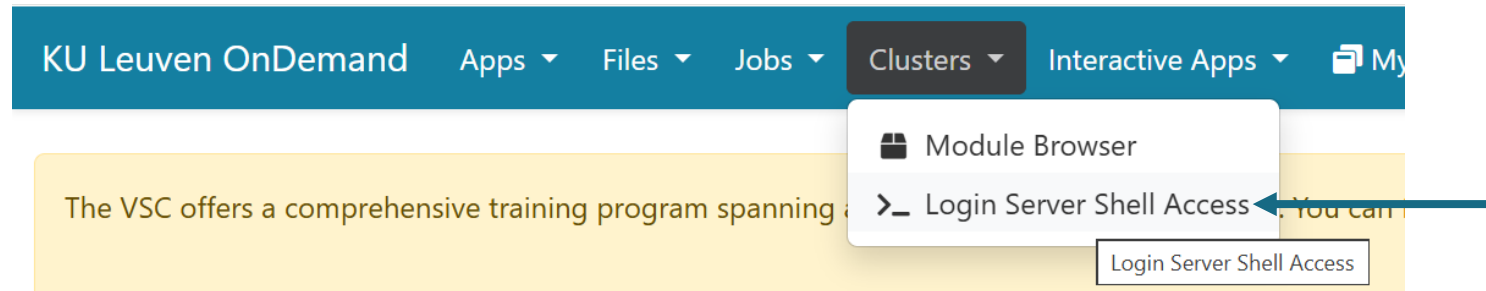
3. During the labs....

*NOTE: copy the commands (orange lines) from the pdf file not from the ppt!

PDF in Toledo: **LAB1_DL_2627_ASSIGNMENT.PDF**

- You have access to code and data via **/lustre1/project/stg_00220 !**

1. Open Terminal



2. Create repository: `mkdir -p "$VSC_DATA/deep_learning/Lab1"`

3. Navigate to your repo: `cd ....`

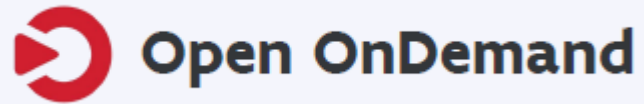
4. Copy .py files into your repo from storage:

`cp -r /lustre1/project/stg_00220/Labs/Lab1/Code .`

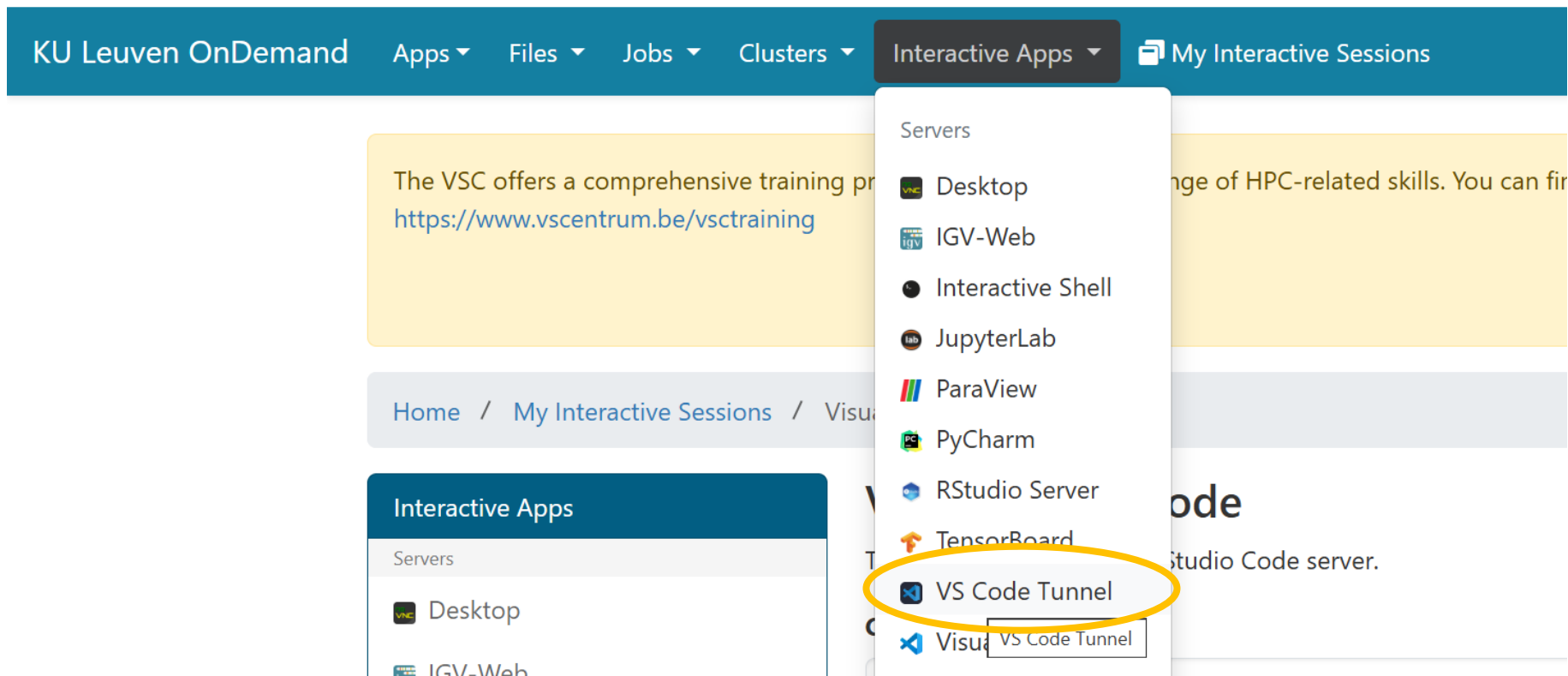
VSC Tunnel

(for coding, debugging on your own laptop)

Open OnDemand



<https://ondemand.hpc.kuleuven.be> Login via KULeuven MFA

A screenshot of the Open OnDemand web interface. The top navigation bar is dark blue with white text for "KU Leuven OnDemand", "Apps", "Files", "Jobs", "Clusters", "Interactive Apps", and "My Interactive Sessions". Below the navigation bar, there are yellow informational boxes. A dropdown menu is open under "Interactive Apps", listing various servers: Desktop, IGV-Web, Interactive Shell, JupyterLab, ParaView, PyCharm, RStudio Server, TensorBoard, VS Code Tunnel (highlighted with a yellow circle), and Visual Studio Code. On the left side, there is a sidebar with a section titled "Interactive Apps" containing a list of servers: Desktop and IGV-Web. The main content area shows a breadcrumb trail "Home / My Interactive Sessions / Visual Studio Code" and a heading "code" followed by the text "Studio Code server."

VS Code Tunnel

Launch a [VS Code tunnel](#) for remote development in your locally installed VS Code application.

Cluster

Mindwell

Partition

interactive

Account

lp_deeplearnig_gt_2026b

[Information about credit account](#)

Number of hours

2

Number of nodes

1

☐ Show advanced options

Launch

* The VS Code Tunnel session data for this session can be accessed under the [data root directory](#).

On KU Leuven Tier-2 clusters, you can request CPU-only resources.

VS Code Tunnel (62086285)

1 node

1 core

Running

Host: >_ k28i14.tier2.hpc.kuleuven.be

✕ Cancel

Created at: 2026-09-21 13:16:41 CEST

Time Remaining: 59 minutes

Session ID: 1958b23f-15ee-429f-aaf0-5203e04ee717

👁 Connect to VS Code Tunnel

- Select & Copy to clipboard from terminal (right-hand mouse)
 - Line: **Left Click & Drag**
 - Block: **Alt+Left Click & Drag**
- Paste in terminal from clipboard
 - **Shift+Insert**
 - **Ctrl+Shift+V**

1. A first browser tab or window opens, showing a terminal session, and you are asked 'How would you like to log in to Visual Studio Code?'.

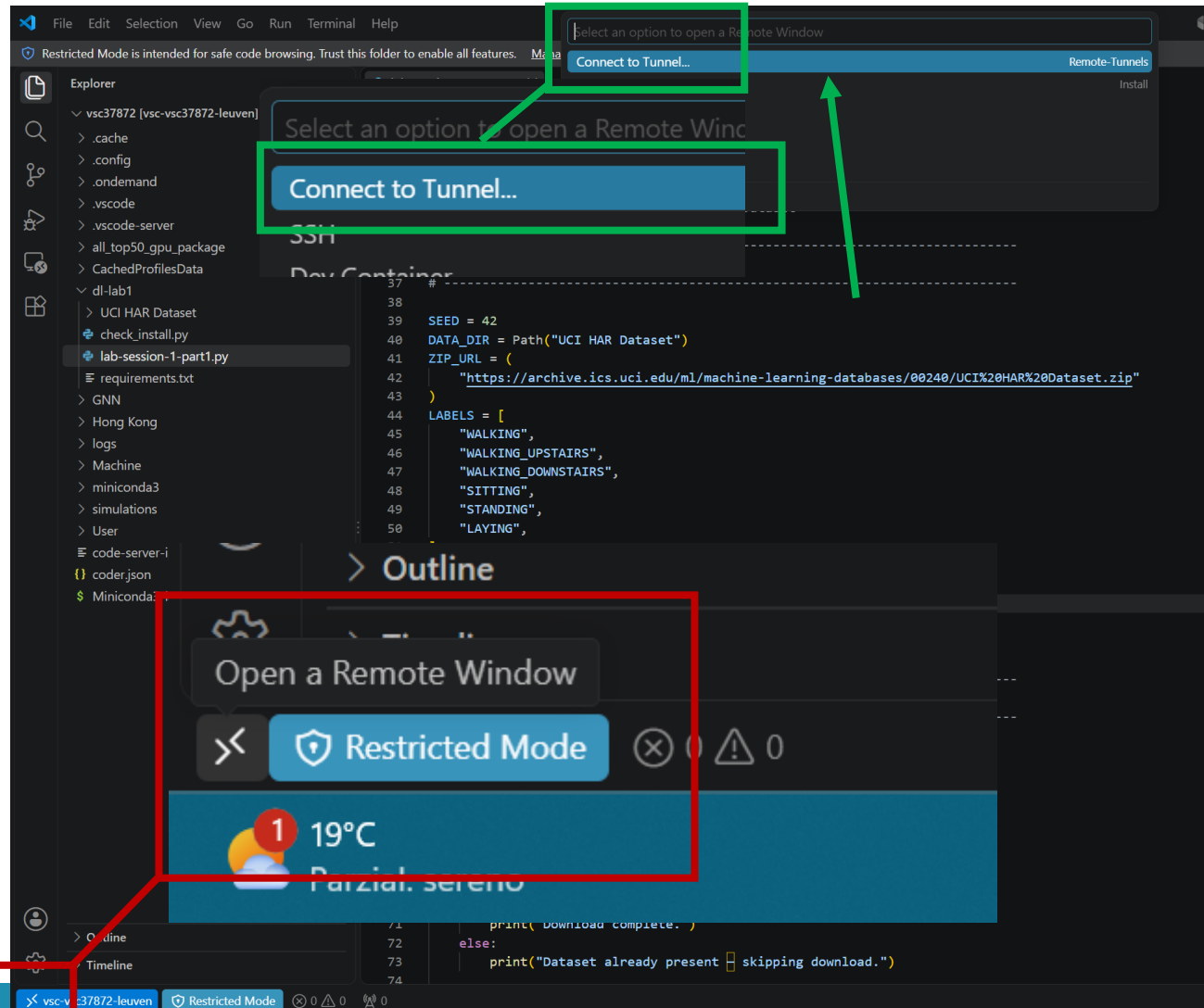
```
export VSCODE_CLI_USE_FILE_KEYCHAIN=1
tmux -S /tmp/J62182547.sock wait-for -S CUSTOMCHANNEL
reset
echo
echo "Copy the device code, open the url and paste it to create the tunnel vsc-vsc37872-wice"
echo
/apps/leuven/common/software/code-cli/1.97.2/code tunnel --accept-server-license-terms --name vsc-vsc37872-leuven
✓ [Sep/29 10:40] vsc37872@k28i14 /data/leuven/378/vsc37872 $ export VSCODE_CLI_USE_FILE_KEYCHAIN=1
✓ [Sep/29 10:40] vsc37872@k28i14 /data/leuven/378/vsc37872 $ tmux -S /tmp/J62182547.sock wait-for -S CUSTOMCHANNEL
✓ [Sep/29 10:40] vsc37872@k28i14 /data/leuven/378/vsc37872 $ reset
✓ [Sep/29 10:40] vsc37872@k28i14 /data/leuven/378/vsc37872 $ echo
✓ [Sep/29 10:40] vsc37872@k28i14 /data/leuven/378/vsc37872 $ echo "Copy the device code, open the url and paste it to create the tunnel vsc-vsc37872-wice"
Copy the device code, open the url and paste it to create the tunnel vsc-vsc37872-wice
✓ [Sep/29 10:40] vsc37872@k28i14 /data/leuven/378/vsc37872 $ echo
✓ [Sep/29 10:40] vsc37872@k28i14 /data/leuven/378/vsc37872 $ /apps/leuven/common/software/code-cli/1.97.2/code tunnel --accept-server-license-terms --name vsc-vsc37872-leuven
*
* Visual Studio Code Server
*
* By using the software, you agree to
* the Visual Studio Code Server License Terms (https://aka.ms/vscode-server-license) and
* the Microsoft Privacy Statement (https://privacy.microsoft.com/en-US/privacystatement).
*
[2026-09-29 10:40:58] info error refreshing token: Error getting authorization: invalid_grant AADSTS50173: The provided grant has expired due to it being revoked, a fresh auth token is needed. The user might have changed or reset their password. The grant was issued on '2026-09-17T08:02:10.0250000Z' and the TokensValidFrom date (before which tokens are not valid) for this user is '2026-09-28T09:08:04.0000000Z'. Trace ID: efc6e0a3-7fb4-440b-998a-da7d0a86b600 Correlation ID: f649c1f0-d67a-411b-9d62-8866663448e0 Timestamp: 2026-09-29 08:40:58Z
To sign in, use a web browser to open the page https://login.microsoft.com/device and enter the code GE78ZGJZG to authenticate.
```



2. Select your institute login.....

Once you accessed your KULeuven account.....

Open VSCode on your laptop

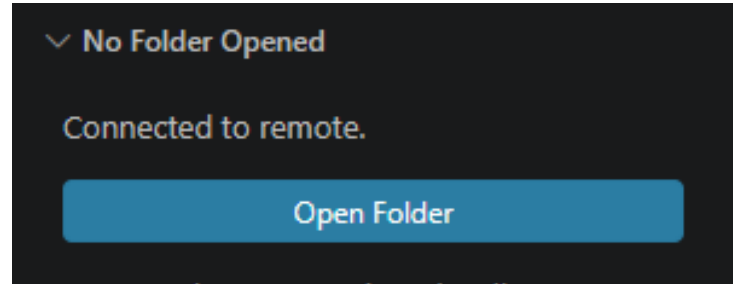


1. click the **button** in the bottom left of the window that says 'Open a Remote Window'.
2. Select 'Tunnel', which will automatically install the 'Remote - Tunnels' extension. If the extension was already installed, click 'Connect to Tunnel..'. ←
3. you are asked 'What type of account did you use to start this tunnel?'. Select 'Microsoft Account'.
4. In the pop-up that says "The extension 'Remote - Tunnels' wants to sign in using Microsoft", click **Allow**.
5. A third browser tab or window opens. Again, select your KU LEUVEN account.
6. Again in the command palette, select the tunnel with the format **vsc-<VSC-id>-<cluster>**, where **<VSC-id>** is your VSC-id, and **<cluster>** is the cluster that you selected in the resources form.

Modules

*NOTE: copy the commands (orange lines) from the pdf file not from the ppt!
PDF in Toledo: **LAB1_DL_2627_ASSIGNMENT.PDF**

- Open folder:



- In terminal:
 - Navigate to your folder
`cd $VSC_DATA/deep_learning/Lab1/`
 - Load modules (a module is a pre-installed software package on the HPC cluster. Instead of installing software yourself, you can load a specific version with module load. This makes the software and its dependencies available in your terminal.)
`module load PyTorch/2.9.1-foss-2025a-CUDA-12.8.0-whl`
 - Run your code in the terminal
`python 01_check_dataset_access.py`

LAB 1 – practical exercises

Open LAB1_DL_2627_ASSIGNMENT.PDF

Running with GPU

(during the LAB)

Terminal – Login Server Shell Access

- `srun -A lp_deeplearning_gt_2026b -M wice -p gpu_a100 -n 1 --gres=shard:1 --reservation=course_reservation --pty bash -l`
- `module load...`
- `cd....`
- `python ....`

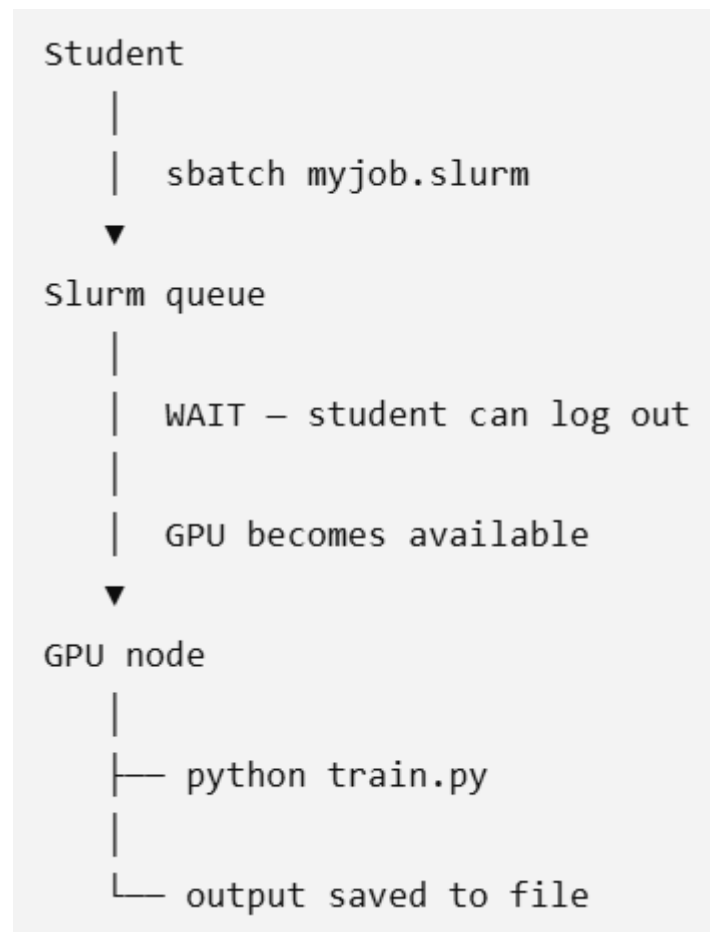
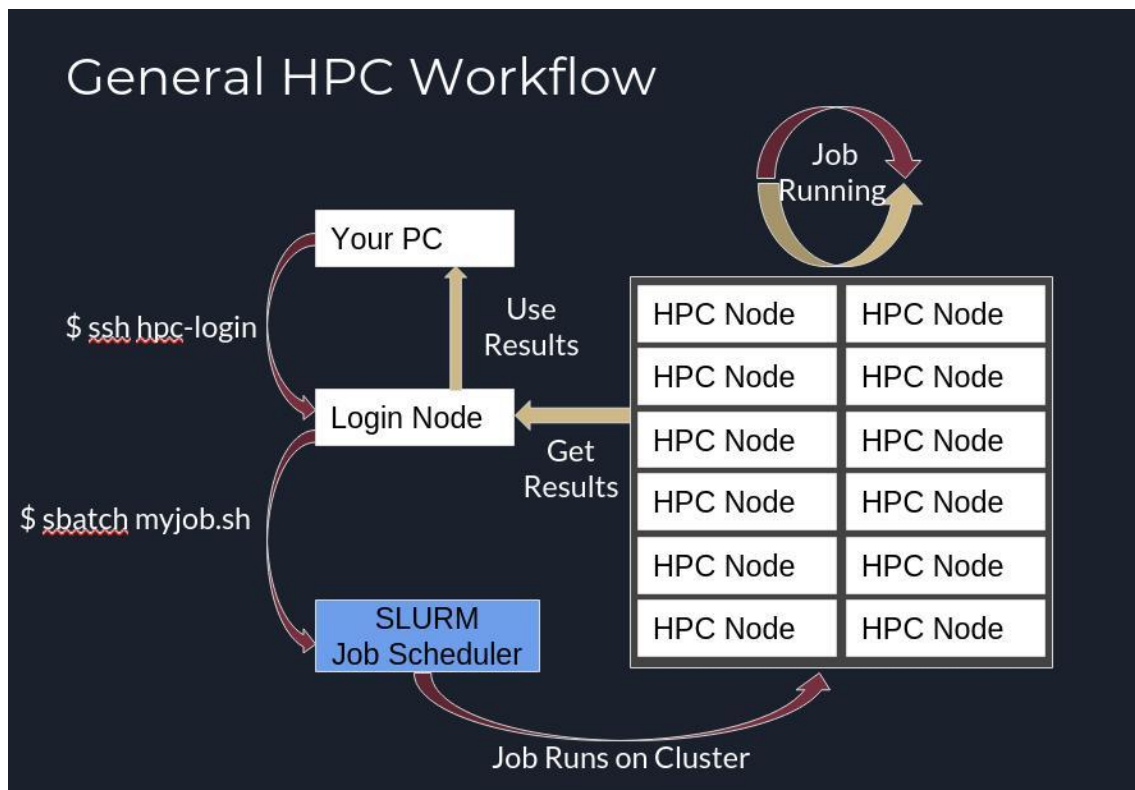
*NOTE: copy the commands (orange lines) from the pdf file not from the ppt!

PDF in Toledo: **LAB1_DL_2627_ASSIGNMENT.PDF**

Running jobs in Slurms

(outside lab hours, GPU)

Core idea - Submit a Slurm batch job with sbatch.



Slurm puts the job in the queue and starts it automatically when the requested resources become available.

Batch Job

You tell Slurm:

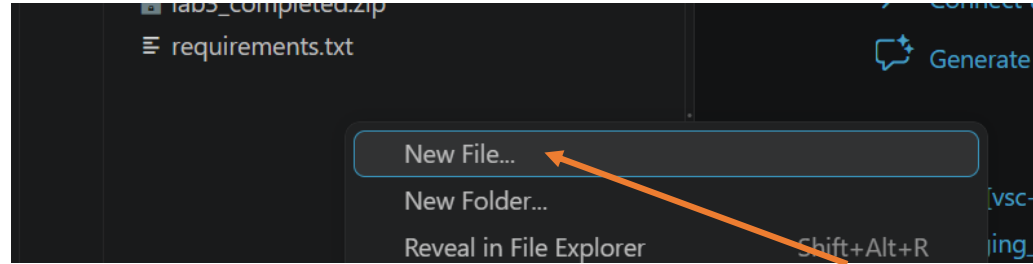
- which cluster you want to use
- which GPU resources you need
- how many CPU cores you need
- how much memory you need
- how long the job may run
- which Python script to execute

Slurm then puts your job in the queue and starts it when the requested resources are available.

This is called a **batch job**.

1. Create slurm file

- Open VSCode



- Example: “run_gpu.slurm”

```
run_gpu.slurm X
run_gpu.slurm
1  #!/bin/bash -l
2  #SBATCH --account=lp_deeplearning_gt_2026b
3  #SBATCH --cluster=wise
4  #SBATCH --partition=gpu_v100
5  #SBATCH --nodes=1 --ntasks=1
6  #SBATCH --gres=shard:1
7  #SBATCH --time=10:00
8  #SBATCH --output=logs-%j.out
9  #SBATCH --error=slurm-%j.err
10
11 module load PyTorch/2.9.1-foss-2025a-CUDA-12.8.0-wh1
12
13 cd $VSC_DATA/dl-lab1
14
15 python lab-session-1-part1.py --skip-demos --skip-plots
16
17
```

Batch Job

- The first line is: `#!/bin/bash -l`
- Specify the resources as needed
- Outside lab hours, remove:
`--gres=shard:1`
- Submit the compute job to the (Slurm) scheduler:
`sbatch example.slurm`
- Slurm will schedule and execute the script without your intervention
- Slurm saves an output file (`logs-<jobID>.out`) in the folder that you submitted your job from

Jobscript `run_gpu.slurm`

```
#!/bin/bash -l
#SBATCH --account=lp_deeplearning_gt_2026b
#SBATCH --cluster=wice
#SBATCH --partition=gpu_v100
#SBATCH --nodes=1 --ntasks=1
#SBATCH --gres=shard:1
#SBATCH --time=10:00
#SBATCH -output=logs-%j.out
#SBATCH -error=slurm-%j.err

module load PyTorch/2.9.1-foss-2025a-CUDA-12.8.0-whl

cd $VSC_DATA/dl-lab1

python lab-session-1-part1.py --skip-demos --skip-plots
```

2. Upload run_gpu.slurm to \$VSC_DATA/dl-lab_1

KU Leuven OnDemand Apps Files Jobs Clusters Interactive Apps

1 Home / Data Directory /data/leuven/378/vsc37872

2 / data / leuven / 378 / vsc37872 / dl-lab1 / Change directory

3 >_Open in Terminal Refresh + New File New Directory Upload Download Open in Globus Copy/Move Delete

4

Type	Name	Size	Modified at
lab-session-1-part1.py	24.96 kB	9/17/2026 11:08:29 AM	
run_gpu.slurm	393.00 B	9/29/2026 10:27:24 AM	

3. Open VSCode Tunnel

VS Code Tunnel (62182547)

1 node | 1 core | Running

Host: `>_ k28il4.tier2.hpc.kuleuven.be`

Created at: 2026-09-29 10:40:36 CEST

Time Remaining: 59 minutes

Session ID: 6de0bb46-9859-4a01-95c4-f274c04bc636

👁 Connect to VS Code Tunnel

- Select & Copy to clipboard from terminal (right-hand mouse)
 - Line: **Left Click & Drag**
 - Block: **Alt+Left Click & Drag**
- Paste in terminal from clipboard
 - **Shift+Insert**
 - **Ctrl+Shift+V**

4. Terminal - your connection to the KU Leuven VSC system

1. Navigate to the folder where your project is: example

```
cd $VSC_DATA/dl-lab1/
```

1. With command **ls** you can see what there is inside your folder: you need to have the .py script you want to run and a **run_gpu.slurm** file
2. **sbatch run_gpu.slurm** : expected output “Submitted batch job 123456”, now you can leave

Check whether your job is waiting

- In terminal: `queue --me`
- PD = The job is in a pending state, waiting for resources
- R = The jobscript is being executed

Thank you !

Q&A